

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

Кафедра ПЗ

МЕТОДИЧНІ ВКАЗІВКИ

до лабораторних робіт
з дисципліни «Комп'ютерна графіка та обробка зображень» для
студентів ОР «Бакалавр»
за спеціальностями F2 Інженерія програмного забезпечення та
F3 Комп'ютерні науки

Методичні вказівки до виконання лабораторних робіт з дисципліни «Комп'ютерна графіка та обробка зображень» для студентів ОР «Бакалавр» за спеціальностями F2 Інженерія програмного забезпечення та F3 Комп'ютерні науки / Укл.: А.В. Пархоменко, А.В. Туленков, М.В. Калініна – Запоріжжя: НУ «Запорізька політехніка», 2025. – 65 с.

Укладачі: А.В. Пархоменко, к.т.н., доцент кафедри ПЗ,
А.В. Туленков, асистент кафедри ПЗ.
М.В. Калініна, асистент кафедри ПЗ.

Рецензент: Н.О.Миронова, к.т.н., доцент кафедри ІТЕЗ

Відповідальний
за випуск: С.О.Субботін, д.т.н., завідувач кафедри ПЗ

Затверджено
на засіданні кафедри
програмних засобів
Протокол № 7
від “23”січня 2025 року

ЗМІСТ

ЗМІСТ	3
1 ЛАБОРАТОРНА РОБОТА №1	5
1.1 Мета циклу робіт	5
1.2 Стислі теоретичні відомості	5
1.3 Налаштування середовища розробки	7
1.4 Завдання на виконання лабораторної роботи	14
2 ЛАБОРАТОРНА РОБОТА №2	15
2.1 Стислі теоретичні відомості.....	15
2.2 Завдання на виконання лабораторної роботи.....	16
3. ЛАБОРАТОРНА РОБОТА №3	17
3.1 Стислі теоретичні відомості.....	17
3.2 Завдання на виконання лабораторної роботи	19
4. ЛАБОРАТОРНА РОБОТА №4	20
4.1 Стислі теоретичні відомості	20
4.2 Завдання на виконання лабораторної роботи	21
5. ЛАБОРАТОРНА РОБОТА №5	22
5.1 Стислі теоретичні відомості	22
5.2 Завдання на виконання лабораторної роботи	23
6. ЛАБОРАТОРНА РОБОТА №6	24
6.1 Стислі теоретичні відомості.....	24
6.2 Завдання на виконання лабораторної роботи	26
7. ЛАБОРАТОРНА РОБОТА №7	27
7.1 Стислі теоретичні відомості.....	27
7.2 Завдання на виконання лабораторної роботи	27
8. ЛАБОРАТОРНА РОБОТА №8	29
8.1 Стислі теоретичні відомості.....	29
8.2 Завдання на виконання лабораторної роботи	29
8.3 Контрольні запитання	30
ДОДАТКИ.....	31
Додаток А – Перша програма.....	31
Додаток Б – Створення шейдерів	32
Додаток В - Переміщення, обертання та масштабування	35
Додаток Д - Використання інтерполяції, індексів малювання та проєкцій.....	39

Додаток Є - Реалізації керування камери.....	45
Додаток Ж - Накладання текстур.....	49
Додаток З – Навколишнє освітлення.....	53
Додаток І – Прожекторне освітлення.....	57
РЕКОМЕНДОВАНІ ДЖЕРЕЛА ІНФОРМАЦІЇ.....	65

1 ЛАБОРАТОРНА РОБОТА №1

1.1 Мета циклу робіт

Отримати практичні навички роботи з бібліотекою OpenGL, використовуючи мову програмування C++. Навчитися малювати 3D об'єкти, виконувати базові маніпуляції з ними, створювати вікна, керувати камерою, працювати з освітленням, створювати прості шейдери та зрозуміти як працюють бібліотеки GLEW, GLFW, GLM.

1.2 Стислі теоретичні відомості

OpenGL (Open Graphics Library) - це крос-платформена багатофункціональна бібліотека для роботи з комп'ютерною графікою. Вона надає програмістам інструменти для створення візуально складних інтерактивних застосунків, включаючи 2D та 3D графіку, анімацію, відео та інші графічні ефекти. OpenGL дозволяє розробникам працювати з різними графічними апаратами та операційними системами безпосередньо, що робить її важливим інструментом для графічного програмування на багатьох платформах.

Основні функції та можливості OpenGL включають:

- рендеринг 2D та 3D графіки: OpenGL надає функції для створення графічних об'єктів, таких як точки, лінії, трикутники та 3D моделі, та їх відображення на екрані;

- робота з текстурами: OpenGL дозволяє накладати текстури на графічні об'єкти, що дозволяє створювати більш деталізовані та реалістичні зображення;

- шейдери: це одна з ключових особливостей OpenGL. Шейдери - це невеликий програмний код, який виконується на графічному процесорі (GPU) і дозволяє здійснювати розрахунки на графічному апараті, що робить можливим створення складних візуальних ефектів;

- підтримка різних операційних систем (ОС): OpenGL підтримується на різних ОС, включаючи Windows, macOS, Linux та інші;

- крос-платформеність: розроблені з використанням OpenGL програми можуть працювати на різних платформах без змін коду.

OpenGL використовується в багатьох областях, таких як відеоігри, комп'ютерні анімації, наукова візуалізація, програми для машинного навчання та багато інших сфер.

GLM" (OpenGL Mathematics), яка є бібліотекою математичних функцій для роботи з 3D графікою та комп'ютерною графікою загалом.

GLM надає функціонал для виконання математичних операцій, таких як трансформації, обертання, масштабування, а також для роботи з векторами та матрицями. Вона розроблена так, щоб бути схожою на інтерфейс і функціонал, які доступні в шейдерах OpenGL, тим самим полегшуючи взаємодію з кодом шейдерів.

GLEW (OpenGL Extension Wrangler Library) - це бібліотека для мови програмування C і C++, яка допомагає розробникам більш просто працювати з розширеннями OpenGL.

GLEW надає розробникам можливість перевіряти, які розширення підтримуються на конкретній системі, і завантажувати функції цих розширень під час виконання програми. Він робить процес роботи з розширеннями OpenGL більш придатним для крос-платформеного програмування, оскільки різні ОС можуть підтримувати різні набори розширень.

GLEW включає в себе генерацію заголовочних файлів, які містять оголошення всіх функцій і констант розширень OpenGL, що підтримуються на певній системі. Це дозволяє програмістам зручно користуватися цими розширеннями у своїх програмах.

GLFW (Graphics Library Framework) - це бібліотека для створення віконних інтерфейсів та обробки подій в графічних програмах (рис. 1.1). Вона спрощує ініціалізацію та управління вікнами, обробку введення від користувача (таку як клавіші і миша) та управління контекстом OpenGL. GLFW спроектована для роботи з OpenGL, але вона також підтримує інші графічні бібліотеки, такі як Vulkan.

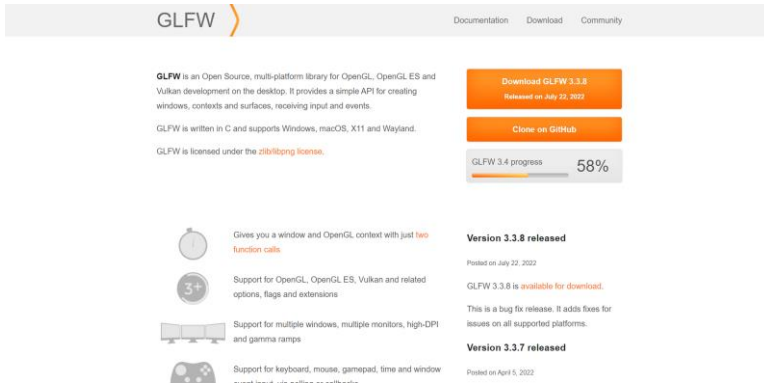


Рисунок 1.1 – Головна сторінка GLFW

1.3 Налаштування середовища розробки

Для того щоб щось створити, спочатку потрібно налаштувати середовище розробки. Переходимо за цим <https://glew.sourceforge.net/> посиланням та завантажуюмо останню версію GLEW – glew-2.1.0-win32.zip (рис. 1.2).

Name ▾	Modified ▾	Size ▾	Downloads / Week ▾
📁 Parent folder			
glew-2.1.0-win32.zip	2017-07-31	2.4 MB	1,742 📄 ⓘ
glew-2.1.0.zip	2017-07-31	1.2 MB	641 📄 ⓘ
glew-2.1.0.tgz	2017-07-31	764.1 kB	194 📄 ⓘ
Totals: 3 items		4.4 MB	2,577

Рисунок 1.2 – Остання версія GLEW

Далі необхідно завантажити GLFW. Для цього ми переходимо за посиланням <https://www.glfw.org/download.html> і завантажуюмо 32-бітну версію (рис. 1.3).

Windows pre-compiled binaries

These packages contain the GLFW header files, [documentation](#) and release mode static libraries, DLLs and import libraries for Visual C++ 2010-2019 and the 2022 preview, MinGW-w64 and plain MinGW.

Binaries for Visual C++ 2010 and plain MinGW are only available in the 32-bit package.

64-bit Windows binaries

32-bit Windows binaries

Рисунок 1.3 – Остання версія GLFW

Додатково необхідно завантажити бібліотеку: <https://glm.g-truc.net/0.9.8/index.html>

Заходимо у Visual Studio та створюємо порожній проєкт (рис. 1.4).

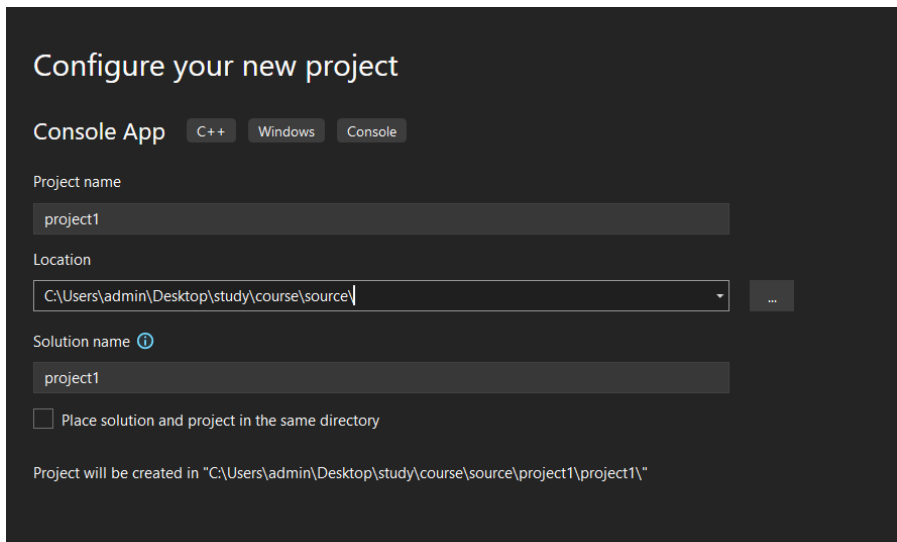


Рисунок 1.4 – Створення нового проєкту

У вихідних файлах створюємо новий файл main.cpp (рис. 1.5).

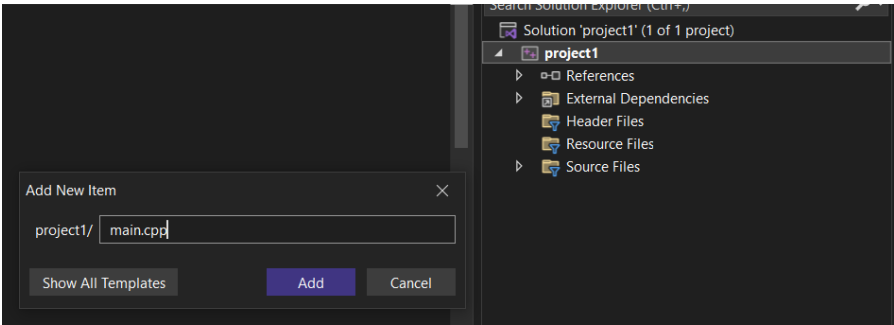


Рисунок 1.5 – Створення main.cpp

Далі заходимо у властивості нашого проєкту. Змінюємо конфігурацію на “Всі конфігурації” та платформу на “Всі платформи” (рис. 1.6).

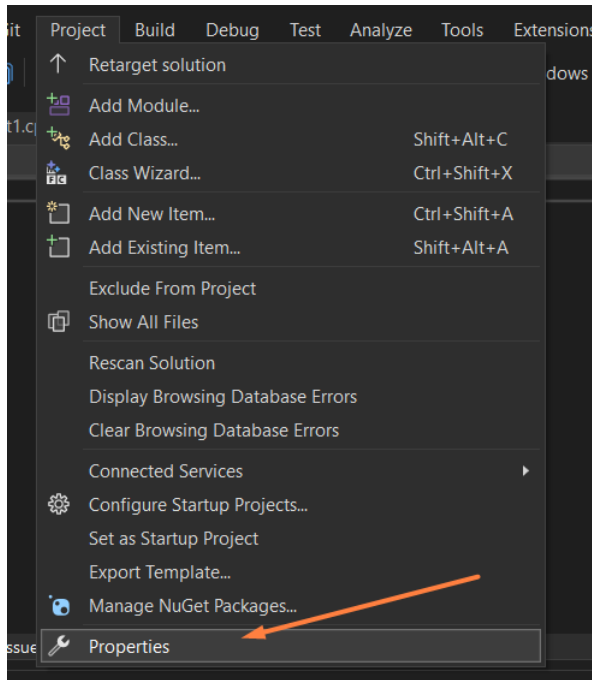


Рисунок 1.6 – Властивості проєкту

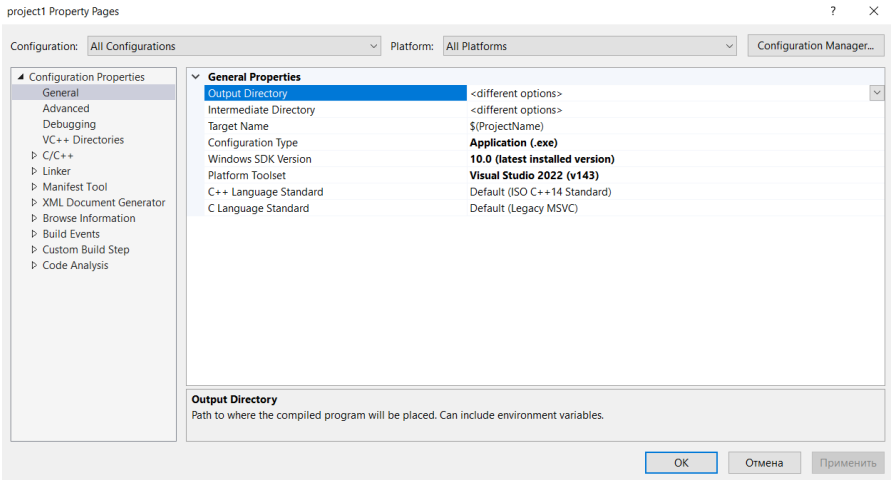


Рисунок 1.7 – Властивості проекту

Потім у розділі “C/C++ -> Загальні” в першому полі вказуємо шлях до наших бібліотек (рис. 1.8-1.10).

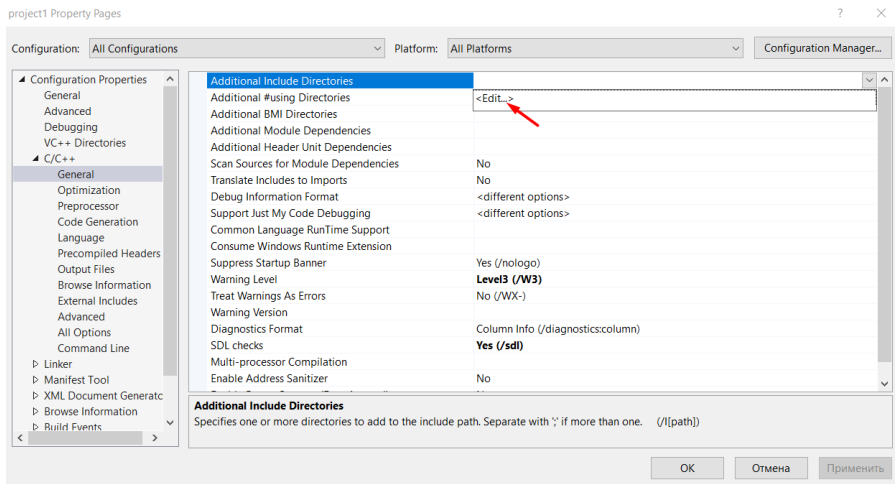


Рисунок 1.8 – Налаштування бібліотек

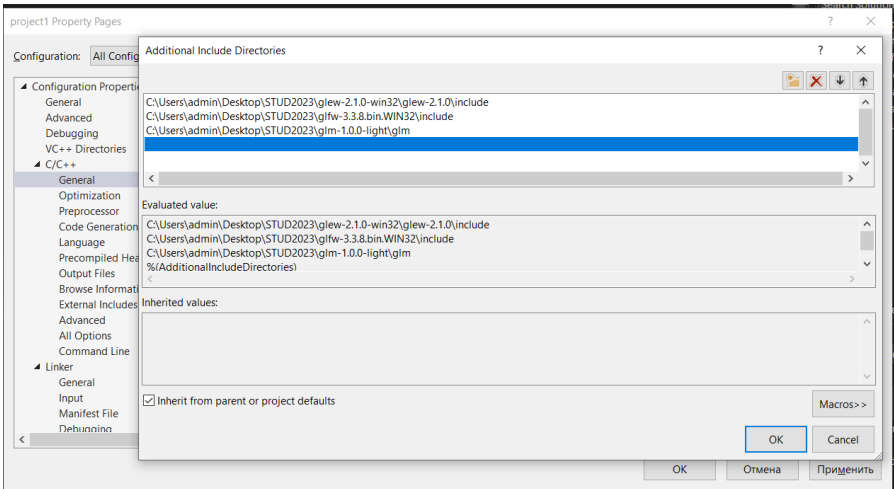


Рисунок 1.9 – Налаштування бібліотек

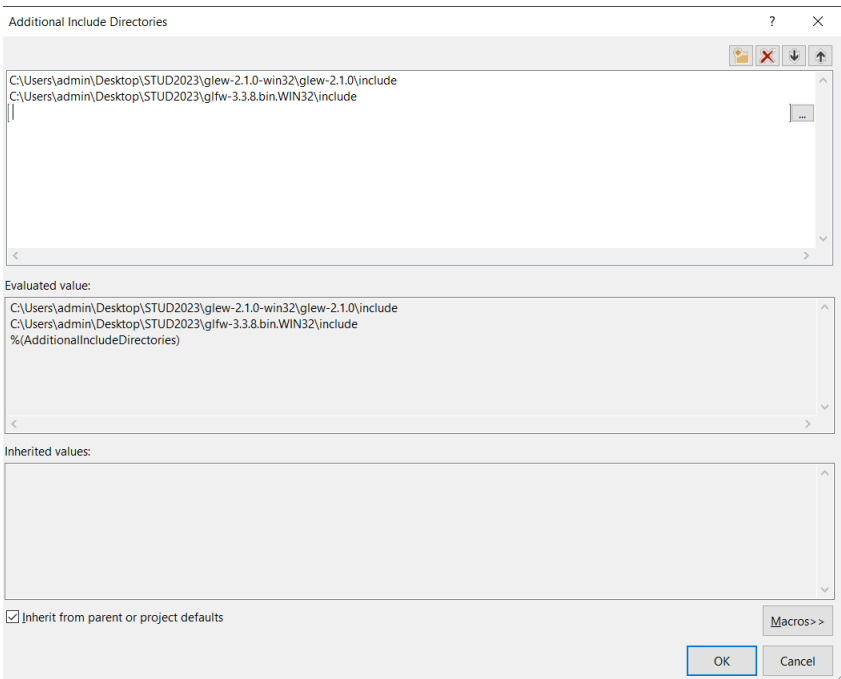


Рисунок 1.10 – Налаштування бібліотек. Шлях до GLEW та GLFW

У розділі “Компонувальник (Linker) -> Загальні -> Додаткові каталоги бібліотек” вказуємо ще один шлях до Win32 та Lib-vc2022 (рис. 1.11).

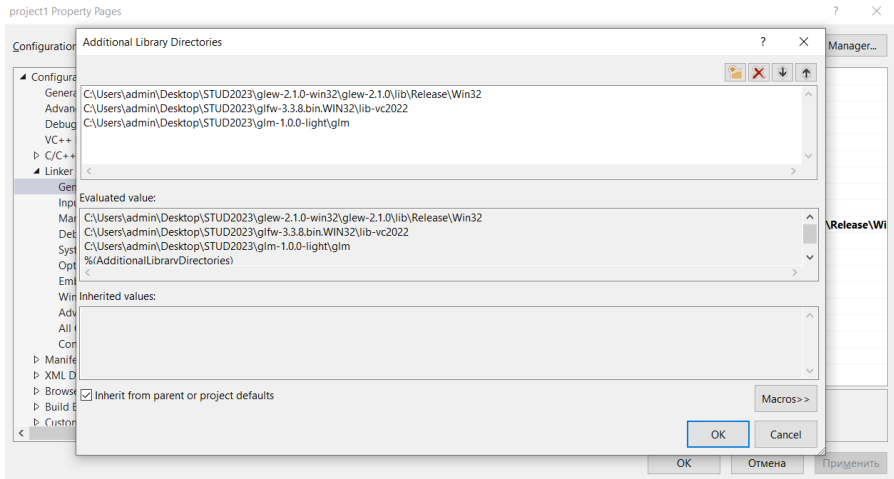


Рисунок 1.11 – Налаштування компонувальника

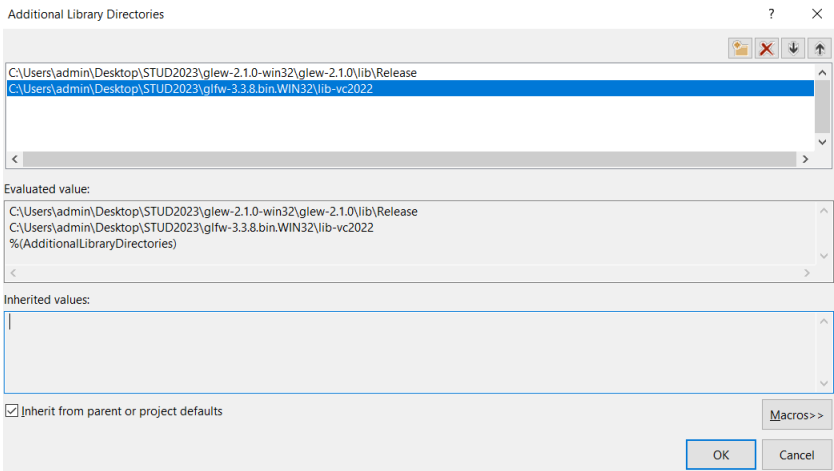


Рисунок 1.12 – Налаштування компонувальника. Шлях до GLEW та GLFW

Далі потрібно перейти у розділ “Компонувальник -> Ввід -> Додаткові залежності” та ввести `opengl32.lib, glew32.lib, glfw3.lib` (рис. 1.13).

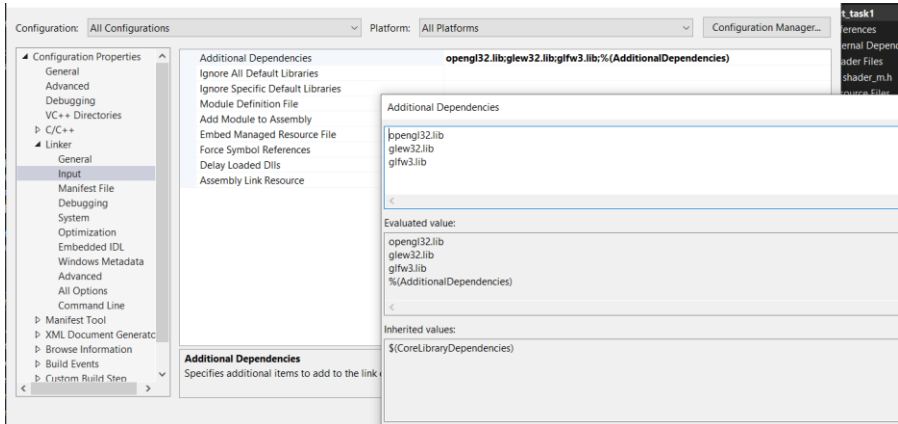


Рисунок 1.13 – Останній крок налаштування

Останній крок, який потрібно зробити, це знайти файл `glew32.dll` (рис. 1.14) у файлах бібліотек **GLEW/bin/Release/Win32** та скопіювати й перенести його у директорію нашого проєкту(`glew32.dll` повинен бути разом з `main.cpp`).

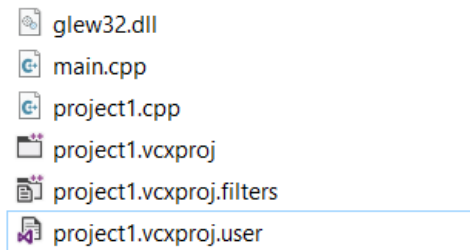


Рисунок 1.14 – Додавання `glew32.dll`

Щоб перевірити, що все працює, необхідно написати просту програму і запустити код (див. Додаток А).

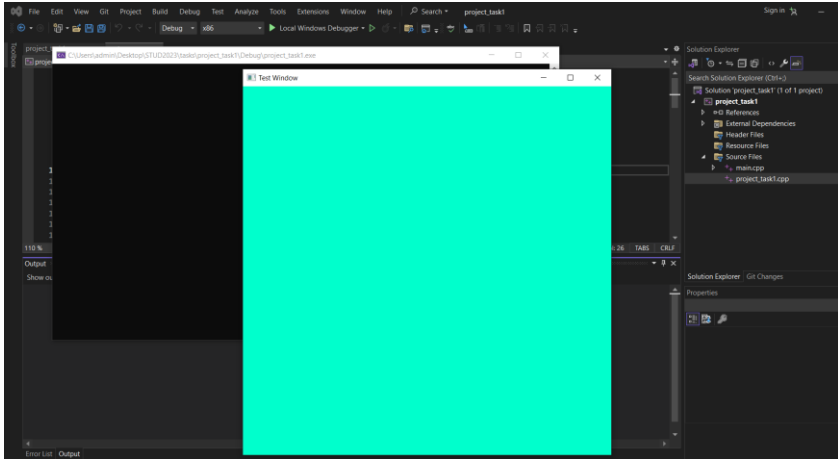


Рисунок 1.5 – Перша програма

1.4 Завдання на виконання лабораторної роботи

1. Змініть колір та розмір вікна (як приклад можна змінити розмір під існуючий пристрій, наприклад планшет або телефон).
2. Наведіть коментарі до коду.
3. Внесіть індивідуальні зміни до коду.

2 ЛАБОРАТОРНА РОБОТА №2

Створення шейдерів

2.1 Стислі теоретичні відомості

Шейдери - це програми, які використовуються у графічному програмуванні для обчислення вигляду й обробки графічних об'єктів. Вони виконуються на графічному процесорі (GPU) та відповідають за рендеринг (відображення) графічних об'єктів на екрані. Шейдери грають ключову роль у відтворенні реалістичних візуальних ефектів у комп'ютерних іграх, анімації, візуалізації та інших графічних застосунках.

Шейдери розділяються на два основних типи:

- вершинні (Vertex Shaders): виконують обчислення для кожної вершини (точки) графічного об'єкта, зазвичай використовуються для перетворення вершин у відповідні координати на екрані, а також для передачі даних іншим шейдерам;

- фрагментні (Fragment Shaders або Pixel Shaders): виконують обчислення для кожного пікселя на екрані після вершинного обчислення, визначають кольори та інші властивості для кожного пікселя на екрані, що дозволяє створювати складні текстури, освітлення та спеціальні ефекти.

Шейдери дозволяють програмістам створювати дуже реалістичні та деталізовані графічні сцени, контролюючи кожен аспект візуального відображення. Вони також використовуються для створення анімацій, обробки зображень, обчислення фізичної моделі освітлення та багатьох інших графічних завдань.

Шейдери використовують спеціальну мову програмування, таку як GLSL (OpenGL Shading Language) у випадку OpenGL або HLSL (High-Level Shading Language) у випадку DirectX, для визначення свого функціоналу. Основна мета шейдерів - оптимізація для обчислень на GPU, щоб забезпечити високу швидкість та реалістичність графіки.

Приклад коду створення простої фігури за допомогою шейдерів наведено у Додатку Б.

Приклад результату роботи програми наведено на рисунку 2.1.

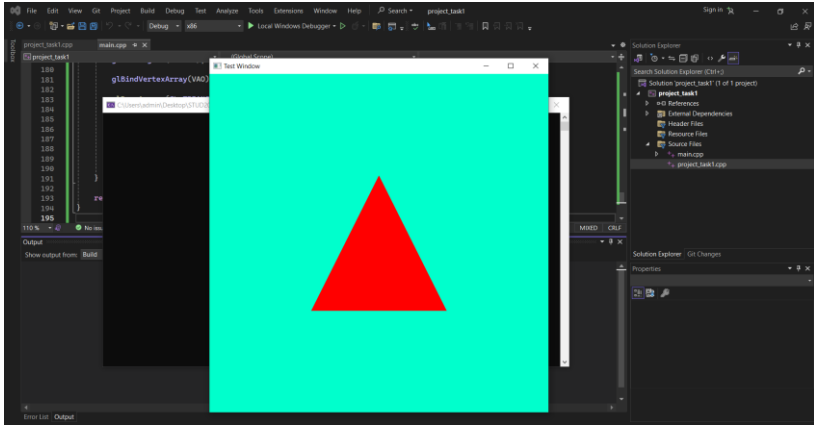


Рисунок 2.1 – Червоний трикутник

2.2 Завдання на виконання лабораторної роботи

1. Змініть фігуру на будь яку іншу нестандартну.
2. Змініть колір фігури та розташування.
3. Наведіть коментарі до коду.
4. Внесіть індивідуальні зміни до коду.

3. ЛАБОРАТОРНА РОБОТА №3

Переміщення, обертання та масштабування

3.1 Стислі теоретичні відомості

Переміщення, обертання та масштабування об'єктів у OpenGL виконується за допомогою матриць трансформацій.

Увага!! Необхідно додати бібліотеку GLM таким чином як попередні бібліотеки.

OpenGL Mathematics (GLM) - це лише заголовок математичної бібліотеки C++ для графічного програмного забезпечення на основі специфікацій OpenGL Shading Language (GLSL). Ця бібліотека ідеально працює з OpenGL, а також забезпечує взаємодію з іншими сторонніми бібліотеками та SDK. Це хороший кандидат для програмного рендерингу (трасування променів / растеризація), обробки зображень, фізичного моделювання та будь-якого контексту розробки, який потребує простої та зручної математичної бібліотеки.

Посилання для завантаження бібліотеки: <https://glm.g-truc.net/0.9.9/index.html> (Розділ Downloads Версія 1.0.0).

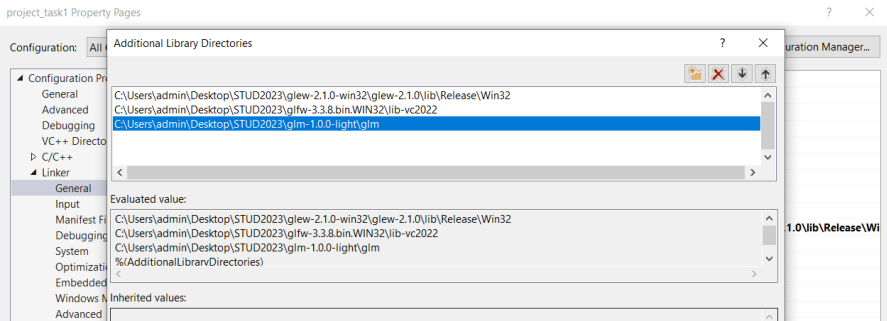


Рисунок 3.1 – Додавання бібліотеки GLM

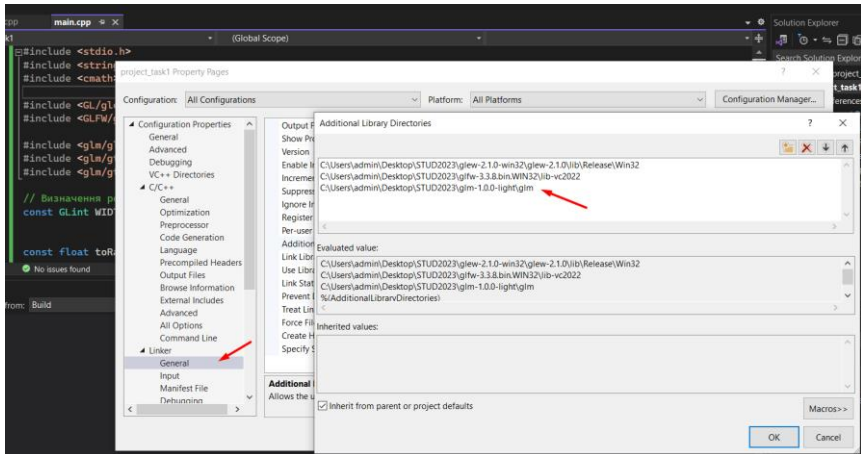


Рисунок 3.2 – Додавання бібліотеки GLM

Для переміщення об'єкта можливо використовувати матрицю моделі (Model Matrix). Ця матриця зсуває об'єкт на вказану відстань у трьох напрямках (x, y, z). Щоб виконати переміщення, потрібно відобразити нове положення об'єкта відносно його поточного положення.

Для обертання об'єкта можливо використовувати матрицю моделі разом із матрицями обертання навколо осей (зазвичай вісь Y). Ці матриці моделі позначають обертання об'єкта навколо визначених осей під кутом. Приклад:

```
glm::mat4 model = glm::mat4(1.0f); // Ініціалізація одиничної матриці моделі
float angle = glm::radians(45.0f); // Кут обертання в радіанах (наприклад, 45 градусів)
model = glm::rotate(model, angle, glm::vec3(0.0f, 1.0f, 0.0f)); // Використання функції rotate
```

Для масштабування об'єкта використовується також матриця моделі. Масштабування відбувається шляхом множення координат об'єкта на відповідні значення масштабу. Приклад:

```
glm::mat4 model = glm::mat4(1.0f); // Ініціалізація одиничної матриці моделі
glm::vec3 scaleVector = glm::vec3(2.0f, 2.0f, 2.0f); // Вектор масштабування
```

```
model = glm::scale(model, scaleVector); // Використання функції scale
```

Після побудови цих матриць моделі їх можливо передавати в шейдер програми для використання в обчисленнях вершин. У шейдерах перемножується вектор вершини на матрицю моделі, щоб відобразити об'єкт з врахуванням трансформації.

Приклад коду переміщення, обертання та масштабування раніше створеної фігури наведено у Додатку В. Результат роботи наведено на рисунку 3.3.

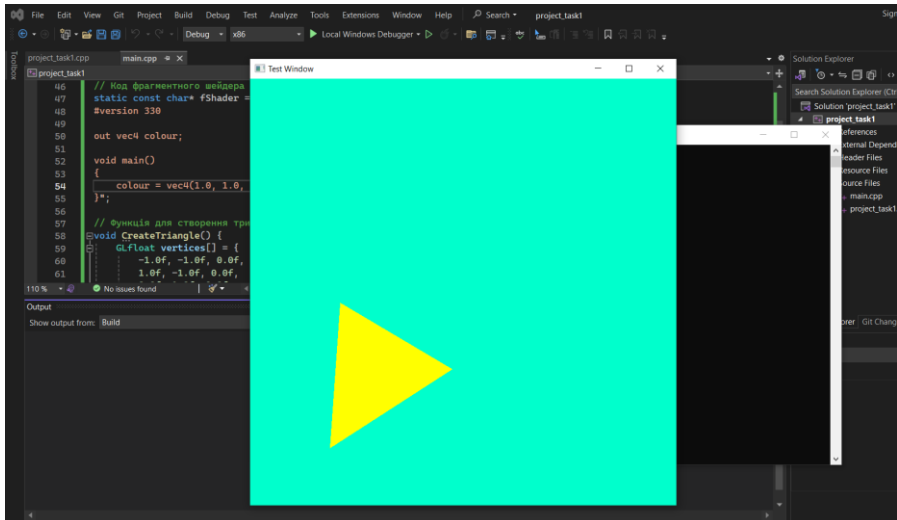


Рисунок 3.3 – Переміщення, обертання та масштабування

3.2 Завдання на виконання лабораторної роботи

1. Змініть колір фону та моделі.
2. Змініть фігуру на будь-яку іншу.
3. Виконайте декілька різних трансформацій з фігурою.
4. Наведіть коментарі до коду.
5. Внесіть індивідуальні зміни до коду.

4. ЛАБОРАТОРНА РОБОТА №4

Інтерполяція, індекси малювання та проєкції

4.1 Стислі теоретичні відомості

Інтерполяція в OpenGL - це процес визначення проміжних значень між двома або більше точками (векторами) на основі заданих значень в початкових точках. Інтерполяція широко використовується у графіці для створення плавних переходів між кольорами, координатами вершин, текстурними координатами та іншими атрибутами. Основні методи інтерполяції включають лінійну інтерполяцію та більш складні методи, такі як сплайн-інтерполяція.

У контексті OpenGL інтерполяція використовується для рендерингу тріангуляції, текстурної інтерполяції, інтерполяції нормалей для визначення освітлення та іншого.

Індекси малювання (англ. Index Buffer або Element Buffer) у OpenGL використовуються для оптимізації роботи з вершинами та підвищення ефективності малювання складних геометричних об'єктів, таких як 3D моделі. Замість повторювання вершин для кожного трикутника або іншої геометричної форми, ми можемо використовувати індексний буфер для визначення, які вершини використовуються для кожного примітиву (трикутника, лінії і т.д.).

Основні кроки використання індексів малювання в OpenGL:

- створення і заповнення буфера індексів: 1) Створіть буфер індексів за допомогою `glGenBuffers`. 2) Зв'яжіть цей буфер з типом `GL_ELEMENT_ARRAY_BUFFER`. 3) Заповніть буфер індексів індексами, які вказують на вершини з масиву вершин;

- вказівка на буфер індексів під час малювання: 1) Використовуйте `glDrawElements` замість `glDrawArrays` для малювання. 2) Вкажіть кількість індексів, які потрібно використовувати під час малювання.

У OpenGL проєкції використовуються для трансформації тривимірних об'єктів і сцен в двовимірний простір, який відображається на екрані. Проєкції важливі для створення реалістичних і зручних для сприйняття зображень. OpenGL надає два

основних типи проєкцій: перспективну та ортографічну (паралельну). Вони використовуються залежно від потреб програми і сцени.

Приклад використання інтерполяції, індексів малювання та проєкцій наведено у Додатку Д. Приклад результату роботи коду наведено на рисунку 4.1.

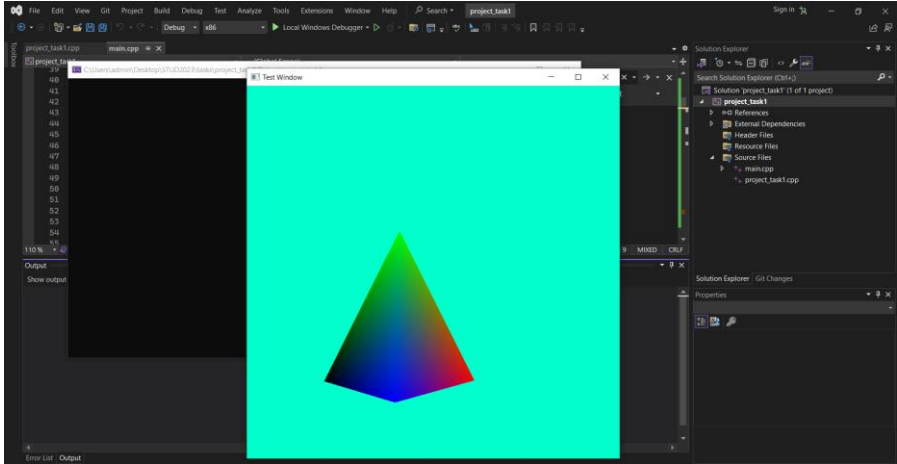


Рисунок 4.1– Кольорова піраміда

4.2 Завдання на виконання лабораторної роботи

1. Змініть фігуру та її колір.
2. Запишіть коротке відео результату.
3. Наведіть коментарі до коду.
4. Внесіть індивідуальні зміни до коду.

5. ЛАБОРАТОРНА РОБОТА №5

Камера

5.1 Стислі теоретичні відомості

Камера в OpenGL відповідає за визначення того, яка частина 3D сцени повинна бути відображена на екрані. Робота з камерою включає налаштування позиції, напрямку, положення та параметрів проєкції. Це дозволяє змінювати точку огляду та область видимості сцени. У OpenGL камера зазвичай реалізується за допомогою матриць моделі, виду і проєкції.

Основні компоненти камери в OpenGL:

- позиція камери: визначає положення камери у світовому (тривимірному) просторі;
- напрямок камери: визначає, куди дивиться камера;
- вектор "вгору" камери: вказує, яка сторона камери вважається вертикальною;
- матриця виду (View Matrix): відповідає за перетворення всіх об'єктів у світі так, щоб вони відображалися відносно камери. Матриця виду визначає позицію і орієнтацію камери;
- матриця проєкції (Projection Matrix): визначає, як камера перетворює тривимірний простір у двовимірний простір, який відображається на екрані. Ця матриця включає параметри, такі як перспектива, область видимості тощо.

Для створення та керування камерою в OpenGL, зазвичай використовують бібліотеки, такі як GLFW або більш високорівневі бібліотеки для графіки.

Приклад реалізації керування камери наведено у Додатку Е. Приклад результату роботи коду наведено на рисунку 5.1.

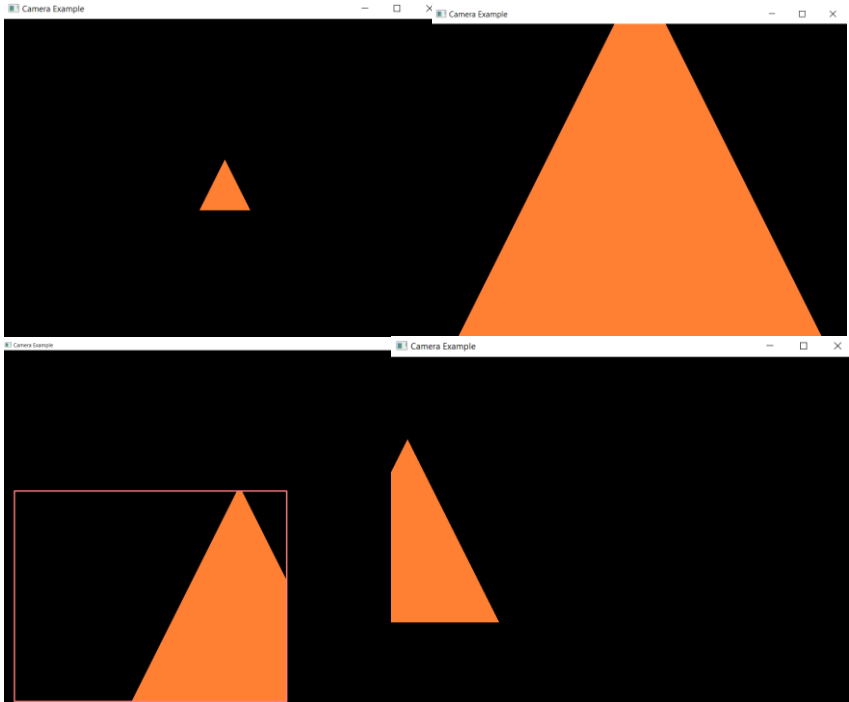


Рисунок 5.1 – Переміщення камери

5.2 Завдання на виконання лабораторної роботи

1. Змініть фігуру та колір.
2. Змініть розмір вікна.
3. Наведіть коментарі до коду.
4. Опишіть алгоритм роботи камери.
5. Знайдіть зайве у коді.
6. Внесіть індивідуальні зміни до коду.

6. ЛАБОРАТОРНА РОБОТА №6

Накладання текстур

6.1 Стислі теоретичні відомості

Накладання текстур у OpenGL - це процес приєднання зображення (текстури) до поверхні 3D моделі. Це дозволяє створювати більш складні та реалістичні візуальні ефекти у графічній програмі.

Робота з текстурами в OpenGL включає наступні кроки:

- завантаження текстурного зображення;
- створення та налаштування текстурного об'єкта;
- налаштування параметрів текстури, таких як режими масштабування й фільтрація;
- завантаження зображення в текстурний об'єкт і генерація міпмап;
- включення текстурного зразка у шейдері та передача ідентифікатора текстури;
- визначення текстурних координат для вершин об'єкта, який текстуриться;
- відтворення об'єкта з текстурою на екрані.

Це загальний підхід до роботи з текстурами в OpenGL, але можливо розширити цей підхід для конкретних потреб програми.

Приклад накладання текстур наведено у Додатку Ж. Приклад результату роботи програми наведено на рисунках 6.1-6.3.

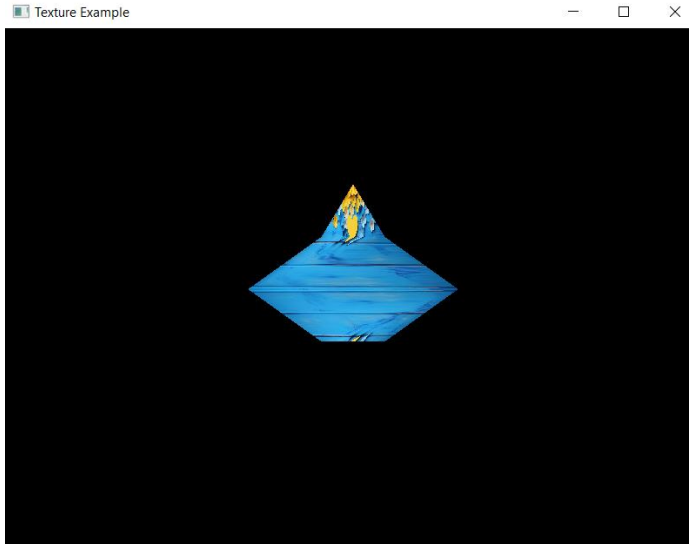


Рисунок 6.1 – Фігура із текстурою

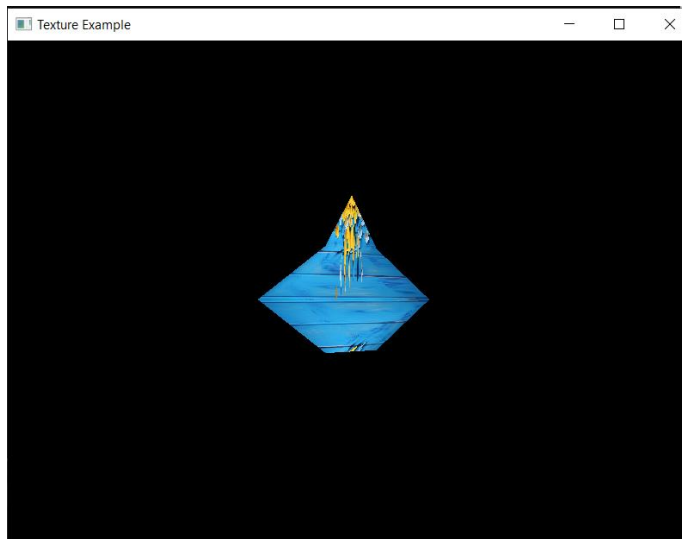


Рисунок 6.2 – Фігура із текстурою з обертанням

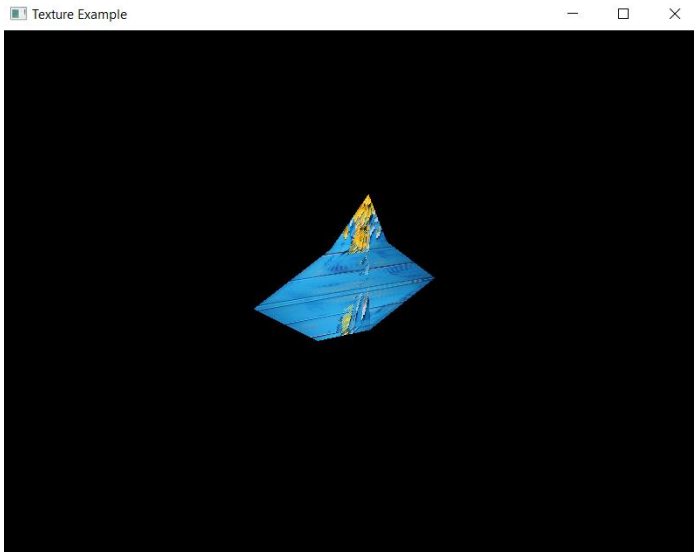


Рисунок 6.3 – Накладання текстур з обертанням

6.2 Завдання на виконання лабораторної роботи

1. Змініть фігуру.
2. Змініть текстуру.
3. Додайте трансформації (наприклад обернути на 45 градусів) фігури із текстурою.
4. Опишіть загальний алгоритм накладання текстури.
5. Наведіть коментарі до коду.
6. Внесіть індивідуальні зміни до коду.

7. ЛАБОРАТОРНА РОБОТА №7

Навколишнє, дифузне та дзеркальне освітлення

7.1 Стислі теоретичні відомості

Навколишнє освітлення (ambient lighting) в OpenGL моделює розсіяване світло, яке надходить з усіх напрямків і освітлює об'єкт рівномірно, незалежно від напрямку та позиції джерела світла. Це додає загальний освітлювальний ефект до сцени та запобігає повному затемненню об'єктів в тіні.

Дифузне освітлення (diffuse lighting) в OpenGL моделює, як світло відбивається від матеріалу об'єкта в усі напрямки. Це дозволяє об'єктам відтворювати тіні та змінювати світлосилу в залежності від кута падіння світла.

Дзеркальне освітлення (specular lighting) в OpenGL моделює блискучість об'єктів. Воно відтворює ефект віддзеркалення світла від гладких поверхонь та створює блискучі плями на матеріалах.

Приклад реалізації навколишнього освітлення наведено у Додатку 3. Приклад дифузного освітлення наведено на рисунку 7.1.

7.2 Завдання на виконання лабораторної роботи

1. Змініть фігуру.
2. Змініть текстуру.
3. Змініть тип освітлення.

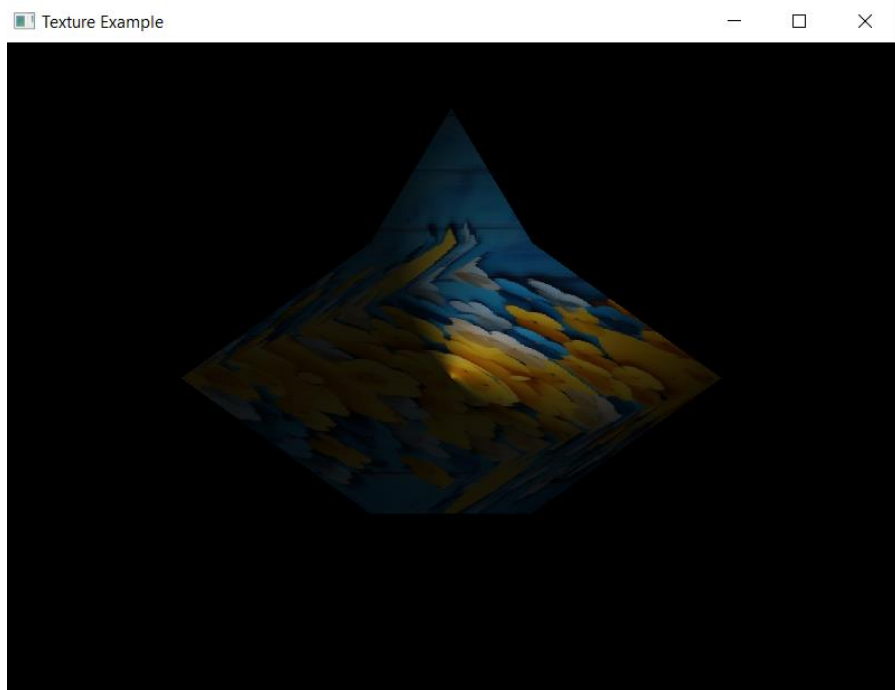


Рисунок 7.1 – Дифузне освітлення

8. ЛАБОРАТОРНА РОБОТА №8

Точкові та прожекторні світильники

8.1 Стислі теоретичні відомості

У OpenGL точкові світильники (Point Lights) та прожекторні світильники (Spotlights) є двома основними типами світлових джерел, які можна використовувати для створення освітлення в 3D-сценах.

Точкові світильники (Point Lights): поширюють світло у всі напрямки з певної точки у просторі. Вони ідеально підходять для симуляції джерел світла, таких як лампи чи свічки.

Прожекторні світильники (Spotlights): випромінюють світло в конкретному напрямку, створюючи променевий конус світла. Вони використовуються для створення спрямованого світла, наприклад, ліхтарів чи фар автомобілів.

Кожен тип світлового джерела має різні параметри, такі як позиція, колір, інтенсивність, відстань впливу та інші, які можна налаштовувати для досягнення бажаного ефекту освітлення у OpenGL-сцені. Використовуючи ці світильники разом з матеріалами та шейдерами, можливо створити реалістичні візуальні ефекти у 3D-програмах та іграх.

Приклад коду наведено у Додатку I. Приклад результату роботи програми наведено на рисунку 8.1.

8.2 Завдання на виконання лабораторної роботи

1. Застосуйте різноманітне освітлення до об'єкта, змініть форму об'єкта.
2. Наведіть алгоритм встановлення освітлення.
3. Наведіть коментарі у коді.

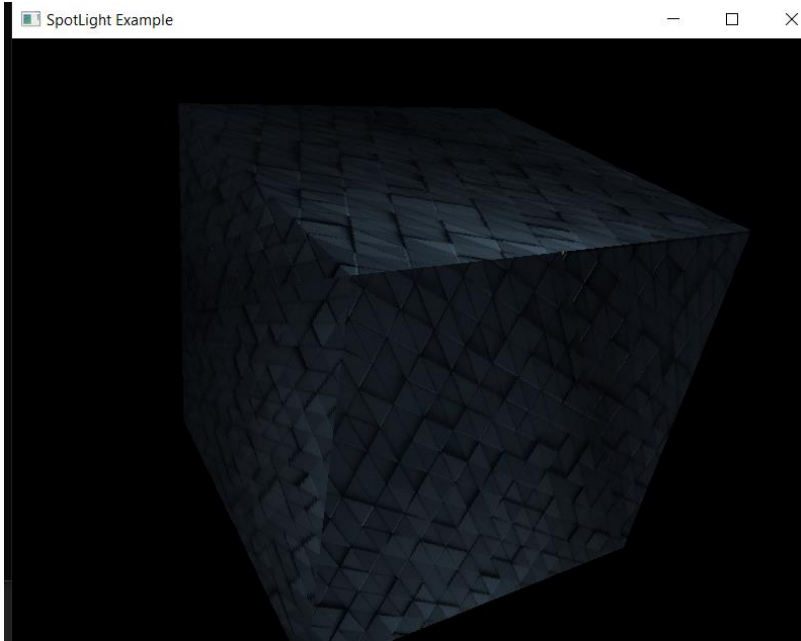


Рисунок 8.1 – Використання освітлення

8.3 Контрольні запитання

- 1.6.1. Що таке OpenGL?
- 1.6.2. Поясніть, що таке GLFW та GLEW.
- 1.6.3. Які типи шейдерів ви знаєте?
- 1.6.4. Як зробити обертання об'єкта?
- 1.6.5. Що таке інтерполяція?
- 1.6.6. Для чого потрібні проєкції в OpenGL?
- 1.6.7. Як працює камера в OpenGL?
- 1.6.8. Поясніть принцип накладання текстур.
- 1.6.9. Що таке UV-розгортка?
- 1.6.10. Що таке навколишнє освітлення?
- 1.6.11. Що таке дифузне освітлення?
- 1.6.12. Що таке дзеркальне освітлення?
- 1.6.13. Поясніть принцип роботи точкового світильника.
- 1.6.14. Поясніть принцип роботи прожекторного світильника.

ДОДАТКИ

Додаток А – Перша програма

Файл main.cpp

```
#include <stdio.h>
#include <GL/glew.h>
#include <GLFW/glfw3.h>
const GLint WIDTH = 800, HEIGHT = 800;
int main() {
    if (!glfwInit())
    {
        printf("GLFW failed to start :(");
        glfwTerminate();
        return 1;
    }
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
    GLFWwindow* mainWindow = glfwCreateWindow(WIDTH, HEIGHT, "Test
Window", NULL, NULL);
    if (!mainWindow)
    {
        printf("GLFW window creation failed :(");
        glfwTerminate();
        return 1;
    }
    int bufferWidth, bufferHeight;
    glfwGetFramebufferSize(mainWindow, &bufferWidth,
&bufferHeight);
    glfwMakeContextCurrent(mainWindow);

    glewExperimental = GL_TRUE;
    if (glewInit() != GLEW_OK)
    {
        printf("GLEW failed to start >_<");
        glfwDestroyWindow(mainWindow);
        glfwTerminate();
        return 1;
    }
    glViewport(0, 0, bufferWidth, bufferHeight);

    while (!glfwWindowShouldClose(mainWindow))
    {
        glfwPollEvents();
        glClearColor(0.0f, 1.0f, 0.8f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT);
        glfwSwapBuffers(mainWindow);
    }
}
```

```

        return 0;
}

```

Додаток Б – Створення шейдерів

Файл main.cpp

```

#include <stdio.h>
#include <string.h>
#include <GL/glew.h>
#include <GLFW/glfw3.h>

const GLint WIDTH = 800, HEIGHT = 800;
GLuint VAO, VBO, shader;
static const char* vShader = "
\n\
#version 330
\n\

\n\
layout (location = 0) in vec3 pos;\n\
\n\
void main()
\n\
{
\n\
    gl_Position = vec4(0.4 * pos.x, 0.4 * pos.y, pos.z, 1.0);
\n\
}";

static const char* fShader = "
\n\
#version 330
\n\
\n\
out vec4 colour;
\n\

\n\
void main()
\n\
{
\n\
    colour = vec4(1.0, 0.0, 0.0, 1.0);
\n\
}";

void CreateTriangle() {

```



```

    GLfloat vertices[] = {
        -1.0f, -1.0f, 0.0f,
        1.0f, -1.0f, 0.0f,
        0.0f, 1.0f, 0.0f
    };

    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);
    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
    glEnableVertexAttribArray(0);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}

void AddShader(GLuint theProgram, const char* shaderCode, GLenum
shaderType)
{
    GLuint theShader = glCreateShader(shaderType);
    const GLchar* theCode[1];
    theCode[0] = shaderCode;
    GLint codeLength[1];
    codeLength[0] = strlen(shaderCode);
    glShaderSource(theShader, 1, theCode, codeLength);
    glCompileShader(theShader);
    GLint result = 0;
    GLchar eLog[1024] = { 0 };
    glGetShaderiv(theShader, GL_COMPILE_STATUS, &result);
    if (!result)
    {
        glGetShaderInfoLog(theShader, 1024, NULL, eLog);
        fprintf(stderr, "Error compiling the %d shader:
%s'\n", shaderType, eLog);
        return;
    }

    glAttachShader(theProgram, theShader);
}

void CompileShaders()
{
    shader = glCreateProgram();
    if (!shader)
    {
        printf("Failed to create shader\n");
        return;
    }
    AddShader(shader, vShader, GL_VERTEX_SHADER);
    AddShader(shader, fShader, GL_FRAGMENT_SHADER);
    GLint result = 0;

```

```

GLchar eLog[1024] = { 0 };
glLinkProgram(shader);
glGetProgramiv(shader, GL_LINK_STATUS, &result);
if (!result)
{
    glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
    printf("Error linking program: '%s'\n", eLog);
    return;
}
glValidateProgram(shader);
glGetProgramiv(shader, GL_VALIDATE_STATUS, &result);
if (!result)
{
    glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
    printf("Error validating program: '%s'\n", eLog);
    return;
}
}

int main() {
    if (!glfwInit())
    {
        printf("GLFW failed to start :(");
        glfwTerminate();
        return 1;
    }
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
    GLFWwindow* mainWindow = glfwCreateWindow(WIDTH, HEIGHT, "Test
Window", NULL, NULL);
    if (!mainWindow)
    {
        printf("GLFW window creation failed :(");
        glfwTerminate();
        return 1;
    }
    int bufferWidth, bufferHeight;
    glfwGetFramebufferSize(mainWindow, &bufferWidth,
&bufferHeight);
    glfwMakeContextCurrent(mainWindow);
    glewExperimental = GL_TRUE;
    if (glewInit() != GLEW_OK)
    {
        printf("GLEW failed to start >_<");
        glfwDestroyWindow(mainWindow);
        glfwTerminate();
        return 1;
    }
    glViewport(0, 0, bufferWidth, bufferHeight);
    CreateTriangle();
    CompileShaders();
    while (!glfwWindowShouldClose(mainWindow))

```

```

{
    glfwPollEvents();
    glClearColor(0.0f, 1.0f, 0.8f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(shader);
    glBindVertexArray(VAO);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glBindVertexArray(0);
    glUseProgram(0);
    glfwSwapBuffers(mainWindow);
}
return 0;
}

```

Додаток В - Переміщення, обертання та масштабування

Файл main.cpp

```

#include <stdio.h>
#include <string.h>
#include <cmath>
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
const GLint WIDTH = 800, HEIGHT = 800;
const float toRadians = 3.14159265f / 180.0f;
float curAngle = 0.0f;
GLuint VAO, VBO, shader, uniformModel;
bool direction = true;
float triOffset = 0.0f;
float triMaxOffset = 0.7f;
float triIncrement = 0.0005f;
bool sizeDirection = true;
float curSize = 0.4f;
float maxSize = 0.8f;
float minSize = 0.1f;
static const char* vShader = "
\n\
#version 330
\n\

\n\
layout (location = 0) in vec3 pos;

\n\
\n\

```

```

uniform mat4 model;
\n\

\n\
void main()
\n\
{
\n\
    gl_Position = model * vec4(pos, 1.0);
\n\
}";
static const char* fShader = "
\n\
#version 330
\n\

\n\
out vec4 colour;
\n\

\n\
void main()
\n\
{
\n\
    colour = vec4(1.0, 0.0, 0.0, 1.0);
\n\
}";
void CreateTriangle() {
    GLfloat vertices[] = {
        -1.0f, -1.0f, 0.0f,
        1.0f, -1.0f, 0.0f,
        0.0f, 1.0f, 0.0f
    };

    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);
    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
    glEnableVertexAttribArray(0);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}
void AddShader(GLuint theProgram, const char* shaderCode, GLenum
shaderType)
{
    GLuint theShader = glCreateShader(shaderType);
    const GLchar* theCode[1];
    theCode[0] = shaderCode;
    GLint codeLength[1];

```

```

    codeLength[0] = strlen(shaderCode);
    glShaderSource(theShader, 1, theCode, codeLength);
    glCompileShader(theShader);
    GLint result = 0;
    GLchar eLog[1024] = { 0 };
    glGetShaderiv(theShader, GL_COMPILE_STATUS, &result);
    if (!result)
    {
        glGetShaderInfoLog(theShader, 1024, NULL, eLog);
        fprintf(stderr, "Error compiling the %d shader:
%s'\n", shaderType, eLog);
        return;
    }

    glAttachShader(theProgram, theShader);
}
void CompileShaders()
{
    shader = glCreateProgram();
    if (!shader)
    {
        printf("Failed to create shader\n");
        return;
    }

    AddShader(shader, vShader, GL_VERTEX_SHADER);
    AddShader(shader, fShader, GL_FRAGMENT_SHADER);
    GLint result = 0;
    GLchar eLog[1024] = { 0 };
    glLinkProgram(shader);
    glGetProgramiv(shader, GL_LINK_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error linking program: '%s'\n", eLog);
        return;
    }
    glValidateProgram(shader);
    glGetProgramiv(shader, GL_VALIDATE_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error validating program: '%s'\n", eLog);
        return;
    }
    uniformModel = glGetUniformLocation(shader, "model");
}

int main() {
    if (!glfwInit())
    {
        printf("GLFW failed to start :(");
        glfwTerminate();
    }
}

```

```

        return 1;
    }
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
    GLFWwindow* mainWindow = glfwCreateWindow(WIDTH, HEIGHT, "Test
Window", NULL, NULL);
    if (!mainWindow)
    {
        printf("GLFW window creation failed :(");
        glfwTerminate();
        return 1;
    }

    int bufferWidth, bufferHeight;
    glfwGetFramebufferSize(mainWindow, &bufferWidth,
&bufferHeight);
    glfwMakeContextCurrent(mainWindow);
    glewExperimental = GL_TRUE;
    if (glewInit() != GLEW_OK)
    {
        printf("GLEW failed to start >_<");
        glfwDestroyWindow(mainWindow);
        glfwTerminate();
        return 1;
    }
    glViewport(0, 0, bufferWidth, bufferHeight);
    CreateTriangle();
    CompileShaders();
    while (!glfwWindowShouldClose(mainWindow))
    {
        glfwPollEvents();
        if (direction)
        {
            triOffset += triIncrement;
        }
        else {
            triOffset -= triIncrement;
        }

        if (abs(triOffset) >= triMaxOffset)
        {
            direction = !direction;
        }
        curAngle += 0.01f;
        if (curAngle >= 360)
        {
            curAngle -= 360;
        }

        if (direction)
        {

```

```

        curSize += 0.0001f;
    }
    else {
        curSize -= 0.0001f;
    }

    if (curSize >= maxSize || curSize <= minSize)
    {
        sizeDirection = !sizeDirection;
    }
    glClearColor(0.0f, 1.0f, 0.8f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
    glUseProgram(shader);
    glm::mat4 model(1.0f);
    model = glm::translate(model, glm::vec3(triOffset,
triOffset, 0.0f));
    model = glm::rotate(model, curAngle * toRadians,
glm::vec3(0.0f, 0.0f, 1.0f));
    model = glm::scale(model, glm::vec3(curSize, curSize,
0.0f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
glm::value_ptr(model));
    glBindVertexArray(VAO);
    glDrawArrays(GL_TRIANGLES, 0, 3);
    glBindVertexArray(0);

    glUseProgram(0);
    glfwSwapBuffers(mainWindow);
}
return 0;
}

```

Додаток Д - Використання інтерполяції, індексів малювання та проєкції

Файл main.cpp

```

#include <stdio.h>
#include <string.h>
#include <cmath>

#include <GL/glew.h>
#include <GLFW/glfw3.h>

#include <glm\glm.hpp>
#include <glm\gtc\matrix_transform.hpp>
#include <glm\gtc\type_ptr.hpp>
const GLint WIDTH = 800, HEIGHT = 800;
const float toRadians = 3.14159265f / 180.0f;

float curAngle = 0.0f;

```

```

GLuint VBO, VAO, IBO, shader, uniformModel, uniformProjection;

bool direction = true;
float triOffset = 0.0f;
float triMaxOffset = 0.7f;
float triIncrement = 0.0002f;

bool sizeDirection = true;
float curSize = 0.4f;
float maxSize = 0.8f;
float minSize = 0.1f;

// Код вершинного шейдера
static const char* vShader = "
\n\
#version 330
\n\

\n\
layout (location = 0) in vec3 pos;

\n\

\n\
out vec4 vCol;

\n\

\n\
uniform mat4 model;
\n\
uniform mat4 projection;
\n\

\n\
void main()
\n\
{
\n\
    gl_Position = projection * model * vec4(pos, 1.0);
\n\
    vCol = vec4(clamp(pos, 0.0f, 1.0f), 1.0);
\n\
}";

// Код фрагментного шейдера
static const char* fShader = "
\n\
#version 330
\n\

\n\

```



```

in vec4 vCol;

\n\

\n\
out vec4 colour;
\n\

\n\
void main()
\n\
{
\n\
    colour = vCol;

\n\

}";

void CreateTriangle() {
    unsigned int indices[] = {
        0, 3, 1,
        1, 3, 2,
        2, 3, 0,
        0, 1, 2
    };
    GLfloat vertices[] = {
        -1.0f, -1.0f, 0.0f,
        0.0f, -1.0f, 1.0f,
        1.0f, -1.0f, 0.0f,
        0.0f, 1.0f, 0.0f
    };
    glGenVertexArrays(1, &VAO);
    glBindVertexArray(VAO);
    glGenBuffers(1, &IBO);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, IBO);
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices),
indices, GL_STATIC_DRAW);

    glGenBuffers(1, &VBO);
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 0, 0);
    glEnableVertexAttribArray(0);
    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
}

void AddShader(GLuint theProgram, const char* shaderCode, GLenum
shaderType)
{
    GLuint theShader = glCreateShader(shaderType);
    const GLchar* theCode[1];
    theCode[0] = shaderCode;

```

```

GLint codeLength[1];
codeLength[0] = strlen(shaderCode);
glShaderSource(theShader, 1, theCode, codeLength);
glCompileShader(theShader);
GLint result = 0;
GLchar eLog[1024] = { 0 };
glGetShaderiv(theShader, GL_COMPILE_STATUS, &result);
if (!result)
{
    glGetShaderInfoLog(theShader, 1024, NULL, eLog);
    fprintf(stderr, "Error compiling the %d shader:
%s'\n", shaderType, eLog);
    return;
}

glAttachShader(theProgram, theShader);
}
void CompileShaders()
{
    shader = glCreateProgram();

    if (!shader)
    {
        printf("Failed to create shader\n");
        return;
    }

    AddShader(shader, vShader, GL_VERTEX_SHADER);
    AddShader(shader, fShader, GL_FRAGMENT_SHADER);
    GLint result = 0;
    GLchar eLog[1024] = { 0 };
    glLinkProgram(shader);
    glGetProgramiv(shader, GL_LINK_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error linking program: '%s'\n", eLog);
        return;
    }
    glValidateProgram(shader);
    glGetProgramiv(shader, GL_VALIDATE_STATUS, &result);
    if (!result)
    {
        glGetProgramInfoLog(shader, sizeof(eLog), NULL, eLog);
        printf("Error validating program: '%s'\n", eLog);
        return;
    }
    uniformModel = glGetUniformLocation(shader, "model");
    uniformProjection = glGetUniformLocation(shader,
"projection");
}
int main() {
    if (!glfwInit())

```

```

{
    printf("GLFW failed to start :(");
    glfwTerminate();
    return 1;
}

glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
GLFWwindow* mainWindow = glfwCreateWindow(WIDTH, HEIGHT, "Test
Window", NULL, NULL);
if (!mainWindow)
{
    printf("GLFW window creation failed :(");
    glfwTerminate();
    return 1;
}

int bufferWidth, bufferHeight;
glfwGetFramebufferSize(mainWindow, &bufferWidth,
&bufferHeight);
glfwMakeContextCurrent(mainWindow);

glewExperimental = GL_TRUE;
if (glewInit() != GLEW_OK)
{
    printf("GLEW failed to start >_<");
    glfwDestroyWindow(mainWindow);
    glfwTerminate();
    return 1;
}

glEnable(GL_DEPTH_TEST);
glViewport(0, 0, bufferWidth, bufferHeight);
CreateTriangle();
CompileShaders();

glm::mat4 projection = glm::perspective(glm::radians(45.0f),
(GLfloat)bufferWidth / (GLfloat)bufferHeight, 0.1f, 100.0f);
while (!glfwWindowShouldClose(mainWindow))
{
    glfwPollEvents();

    if (direction)
    {
        triOffset += triIncrement;
    }
    else {
        triOffset -= triIncrement;
    }

    if (abs(triOffset) >= triMaxOffset)
    {

```

```

        direction = !direction;
    }

    curAngle += 0.01f;
    if (curAngle >= 360)
    {
        curAngle -= 360;
    }

    if (direction)
    {
        curSize += 0.0001f;
    }
    else {
        curSize -= 0.0001f;
    }

    if (curSize >= maxSize || curSize <= minSize)
    {
        sizeDirection = !sizeDirection;
    }

    // Очистка буфера кольору зеленим кольором
    glClearColor(0.0f, 1.0f, 0.8f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glUseProgram(shader);

    glm::mat4 model(1.0f);
    model = glm::translate(model, glm::vec3(triOffset,
    triOffset, -3.0f));
    model = glm::rotate(model, curAngle * toRadians,
    glm::vec3(0.0f, 1.0f, 0.0f));
    model = glm::scale(model, glm::vec3(0.5f, 0.5f,
    0.5f));

    glUniformMatrix4fv(uniformModel, 1, GL_FALSE,
    glm::value_ptr(model));
    glUniformMatrix4fv(uniformProjection, 1, GL_FALSE,
    glm::value_ptr(projection));
    glBindVertexArray(VAO);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, IBO);
    glDrawElements(GL_TRIANGLES, 12, GL_UNSIGNED_INT, 0);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);
    glBindVertexArray(0);
    glUseProgram(0);
    glfwSwapBuffers(mainWindow);
}

return 0;
}

```

Додаток Є - Реалізації керування камери

Файл main.cpp

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

class Camera {
public:
    Camera();
    glm::mat4 getViewMatrix();
    void processInput(GLFWwindow* window);

private:
    glm::vec3 cameraPos;
    glm::vec3 cameraFront;
    glm::vec3 cameraUp;
    float deltaTime;
    float lastFrame;
};

Camera::Camera()
    : cameraPos(glm::vec3(0.0f, 0.0f, 3.0f)),
    cameraFront(glm::vec3(0.0f, 0.0f, -1.0f)),
    cameraUp(glm::vec3(0.0f, 1.0f, 0.0f)), deltaTime(0.0f),
    lastFrame(0.0f) {}

glm::mat4 Camera::getViewMatrix() {
    return glm::lookAt(cameraPos, cameraPos + cameraFront,
        cameraUp);
}

void Camera::processInput(GLFWwindow* window) {
    float currentFrame = glfwGetTime();
    deltaTime = currentFrame - lastFrame;
    lastFrame = currentFrame;

    const float cameraSpeed = 2.5f * deltaTime;
    if (glfwGetKey(window, GLFW_KEY_W) == GLFW_PRESS)
        cameraPos += cameraSpeed * cameraFront;
    if (glfwGetKey(window, GLFW_KEY_S) == GLFW_PRESS)
        cameraPos -= cameraSpeed * cameraFront;
    if (glfwGetKey(window, GLFW_KEY_A) == GLFW_PRESS)
```

```

        cameraPos -= glm::normalize(glm::cross(cameraFront,
cameraUp)) * cameraSpeed;
        if (glfwGetKey(window, GLFW_KEY_D) == GLFW_PRESS)
            cameraPos += glm::normalize(glm::cross(cameraFront,
cameraUp)) * cameraSpeed;
    }

int main() {
    if (!glfwInit()) {
        fprintf(stderr, "GLFW initialization failed\n");
        return -1;
    }

    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE,
GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);

    GLFWwindow* window = glfwCreateWindow(800, 600, "Camera
Example", NULL, NULL);
    if (!window) {
        fprintf(stderr, "GLFW window creation failed\n");
        glfwTerminate();
        return -1;
    }

    glfwMakeContextCurrent(window);

    if (glewInit() != GLEW_OK) {
        fprintf(stderr, "GLEW initialization failed\n");
        glfwDestroyWindow(window);
        glfwTerminate();
        return -1;
    }

    glEnable(GL_DEPTH_TEST);
    glViewport(0, 0, 800, 600);

    // Shader source code
    const char* vertexShaderSource = R"(
        #version 330 core
        layout (location = 0) in vec3 aPos;
        uniform mat4 model;
        uniform mat4 view;
        uniform mat4 projection;
        void main() {

```

```

        gl_Position = projection * view * model *
vec4(aPos, 1.0);
    }
}";

const char* fragmentShaderSource = R"(
    #version 330 core
    out vec4 FragColor;
    void main() {
        FragColor = vec4(1.0, 0.5, 0.2, 1.0);
    }
)";

// Compile and link shaders
GLuint vertexShader = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShader, 1, &vertexShaderSource,
NULL);
glCompileShader(vertexShader);

GLuint fragmentShader =
glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource,
NULL);
glCompileShader(fragmentShader);

GLuint shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
glUseProgram(shaderProgram);

glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);

// Vertex data
GLfloat vertices[] = {
    -0.5f, -0.5f, 0.0f,
    0.5f, -0.5f, 0.0f,
    0.0f, 0.5f, 0.0f
};

// Vertex array and buffer objects
GLuint VAO, VBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);

glBindVertexArray(VAO);

```

```

    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 3 *
sizeof(GLfloat), (GLvoid*)0);
    glEnableVertexAttribArray(0);

    glBindBuffer(GL_ARRAY_BUFFER, 0);
    glBindVertexArray(0);

    // Uniform locations
    GLint modelLoc = glGetUniformLocation(shaderProgram,
"model");
    GLint viewLoc = glGetUniformLocation(shaderProgram,
"view");
    GLint projLoc = glGetUniformLocation(shaderProgram,
"projection");

    Camera camera;

    while (!glfwWindowShouldClose(window)) {
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
        camera.processInput(window);
        glm::mat4 model = glm::mat4(1.0f);
        glm::mat4 view = camera.getViewMatrix();
        glm::mat4 projection =
glm::perspective(glm::radians(45.0f), 800.0f / 600.0f, 0.1f,
100.0f);
        glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
glm::value_ptr(model));
        glUniformMatrix4fv(viewLoc, 1, GL_FALSE,
glm::value_ptr(view));
        glUniformMatrix4fv(projLoc, 1, GL_FALSE,
glm::value_ptr(projection));
        glUseProgram(shaderProgram);
        glBindVertexArray(VAO);
        glDrawArrays(GL_TRIANGLES, 0, 3);
        glBindVertexArray(0);

        glfwSwapBuffers(window);
        glfwPollEvents();
    }

    glfwDestroyWindow(window);
    glfwTerminate();

```



```

    return 0;
}

```

Додаток Ж - Накладання текстур

Файл main.cpp

```

#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>
unsigned int loadTexture(const char* path) {
    unsigned int textureID;
    glGenTextures(1, &textureID);

    int width, height, nrChannels;
    unsigned char* data = stbi_load(path, &width, &height,
&nrChannels, 0);
    if (data) {
        GLenum format;
        if (nrChannels == 1)
            format = GL_RED;
        else if (nrChannels == 3)
            format = GL_RGB;
        else if (nrChannels == 4)
            format = GL_RGBA;

        glBindTexture(GL_TEXTURE_2D, textureID);
        glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0,
format, GL_UNSIGNED_BYTE, data);
        glGenerateMipmap(GL_TEXTURE_2D);

        stbi_image_free(data);
    }
    else {
        stbi_image_free(data);
    }

    return textureID;
}

int main() {

    if (!glfwInit()) {
        fprintf(stderr, "GLFW initialization failed\n");
        return -1;
    }

```

```

glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
GLFWwindow* window = glfwCreateWindow(800, 600, "Texture
Example", NULL, NULL);
if (!window) {
    fprintf(stderr, "GLFW window creation failed\n");
    glfwTerminate();
    return -1;
}
glfwMakeContextCurrent(window);
if (glewInit() != GLEW_OK) {
    fprintf(stderr, "GLEW initialization failed\n");
    glfwDestroyWindow(window);
    glfwTerminate();
    return -1;
}

glEnable(GL_DEPTH_TEST);
glViewport(0, 0, 800, 600);
const char* vertexShaderSource = R"(
    #version 330 core
    layout (location = 0) in vec3 aPos;
    layout (location = 1) in vec2 aTexCoord;

    out vec2 TexCoord;

    uniform mat4 model;
    uniform mat4 view;
    uniform mat4 projection;

    void main() {
        gl_Position = projection * view * model * vec4(aPos,
1.0);
        TexCoord = aTexCoord;
    }
)";

// Шейдерный код фрагментного шейдера
const char* fragmentShaderSource = R"(
    #version 330 core
    out vec4 FragColor;

    in vec2 TexCoord;
    uniform sampler2D texture1;

    void main() {
        FragColor = texture(texture1, TexCoord);
    }
)";
GLuint vertexShader = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);

```

```

glCompileShader(vertexShader);

GLuint fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);

GLuint shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
glUseProgram(shaderProgram);
glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);

GLfloat vertices[] = {
    0.0f,  0.5f, 0.0f,  0.5f, 1.0f,
   -0.15f, 0.25f, 0.0f,  0.0f, 0.5f,
    0.15f,  0.25f, 0.0f,  1.0f, 0.5f,
   -0.5f,  0.0f, 0.0f,  0.0f, 0.0f,
    0.5f,  0.0f, 0.0f,  1.0f, 0.0f,
   -0.15f, -0.25f, 0.0f,  0.0f, 0.5f,
    0.15f, -0.25f, 0.0f,  1.0f, 0.5f,
    0.0f, -0.5f, 0.0f,  0.5f, 0.0f,
};

GLuint indices[] = {
    0, 1, 2, 3, 4, 5, 6, 0, 6
};

GLuint VAO, VBO, EBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glGenBuffers(1, &EBO);

glBindVertexArray(VAO);

glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);

glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 5 *
sizeof(GLfloat), (GLvoid*)0);
glEnableVertexAttribArray(0);

glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 5 *
sizeof(GLfloat), (GLvoid*)(3 * sizeof(GLfloat)));
glEnableVertexAttribArray(1);
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);

```

```

unsigned int texture;
glGenTextures(1, &texture);
glBindTexture(GL_TEXTURE_2D, texture);

glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

int width, height, nrChannels;
stbi_set_flip_vertically_on_load(true);
unsigned char* data = stbi_load("texture.jpg", &width, &height,
&nrChannels, 0);
if (data) {
    GLenum format;
    if (nrChannels == 1)
        format = GL_RED;
    else if (nrChannels == 3)
        format = GL_RGB;
    else if (nrChannels == 4)
        format = GL_RGBA;

    glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0,
format, GL_UNSIGNED_BYTE, data);
    glGenerateMipmap(GL_TEXTURE_2D);

    stbi_image_free(data);
}
else {
    stbi_image_free(data);
}

while (!glfwWindowShouldClose(window)) {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glm::mat4 model = glm::mat4(1.0f);
    glm::mat4 view = glm::lookAt(glm::vec3(0.0f, 0.0f, 3.0f),
glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f));
    glm::mat4 projection = glm::perspective(glm::radians(45.0f),
800.0f / 600.0f, 0.1f, 100.0f);

    GLuint modelLoc = glGetUniformLocation(shaderProgram,
"model");
    GLuint viewLoc = glGetUniformLocation(shaderProgram, "view");
    GLuint projectionLoc = glGetUniformLocation(shaderProgram,
"projection");

    glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
glm::value_ptr(model));
    glUniformMatrix4fv(viewLoc, 1, GL_FALSE,
glm::value_ptr(view));
    glUniformMatrix4fv(projectionLoc, 1, GL_FALSE,
glm::value_ptr(projection));

```

```

    glUseProgram(shaderProgram);
    glBindVertexArray(VAO);
    glBindTexture(GL_TEXTURE_2D, texture);
    glDrawElements(GL_TRIANGLES_STRIP, 9, GL_UNSIGNED_INT, 0);
    glBindVertexArray(0);
    glfwSwapBuffers(window);
    glfwPollEvents();
}
glDeleteVertexArrays(1, &VAO);
glDeleteBuffers(1, &VBO);
glDeleteBuffers(1, &EBO);
glfwDestroyWindow(window);
glfwTerminate();

return 0;
}

```

Додаток 3 – Навколишнє освітлення

Файл main.cpp

```

#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

unsigned int loadTexture(const char* path) {
    unsigned int textureID;
    glGenTextures(1, &textureID);

    int width, height, nrChannels;
    unsigned char* data = stbi_load(path, &width, &height,
    &nrChannels, 0);
    if (data) {
        GLenum format;
        if (nrChannels == 1)
            format = GL_RED;
        else if (nrChannels == 3)
            format = GL_RGB;
        else if (nrChannels == 4)
            format = GL_RGBA;

        glBindTexture(GL_TEXTURE_2D, textureID);
        glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0,
        format, GL_UNSIGNED_BYTE, data);
    }
}

```

```

        glGenerateMipmap(GL_TEXTURE_2D);

        stbi_image_free(data);
    }
    else {
        stbi_image_free(data);
    }

    return textureID;
}

int main() {
    if (!glfwInit()) {
        fprintf(stderr, "GLFW initialization failed\n");
        return -1;
    }
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
    GLFWwindow* window = glfwCreateWindow(800, 600, "Texture
Example", NULL, NULL);
    if (!window) {
        fprintf(stderr, "GLFW window creation failed\n");
        glfwTerminate();
        return -1;
    }
    glfwMakeContextCurrent(window);
    if (glewInit() != GLEW_OK) {
        fprintf(stderr, "GLEW initialization failed\n");
        glfwDestroyWindow(window);
        glfwTerminate();
        return -1;
    }

    glEnable(GL_DEPTH_TEST);
    glViewport(0, 0, 800, 600);
    const char* vertexShaderSource = R"(
        #version 330 core
        layout (location = 0) in vec3 aPos;
        layout (location = 1) in vec2 aTexCoord;

        out vec2 TexCoord;

        uniform mat4 model;
        uniform mat4 view;
        uniform mat4 projection;

        out vec3 FragPos;
        out vec3 Normal;

        void main() {

```

```

        gl_Position = projection * view * model * vec4(aPos,
1.0);

        TexCoord = aTexCoord;

        FragPos = vec3(model * vec4(aPos, 1.0));
        Normal = mat3(transpose(inverse(model))) * aPos;
    }
}";
const char* fragmentShaderSource = R"(
    #version 330 core
    out vec4 FragColor;

    in vec2 TexCoord;
    in vec3 FragPos;
    in vec3 Normal;

    uniform sampler2D texture1;
    uniform vec3 lightPos;
    uniform vec3 viewPos;

    void main() {
        float ambientStrength = 0.1;
        vec3 ambient = ambientStrength * vec3(texture(texture1,
TexCoord));

        vec3 norm = normalize(Normal);
        vec3 lightDir = normalize(lightPos - FragPos);
        float diff = max(dot(norm, lightDir), 0.0);
        vec3 diffuse = diff * vec3(texture(texture1, TexCoord));

        vec3 result = (ambient + diffuse);
        FragColor = vec4(result, 1.0);
    }
}";
GLuint vertexShader = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
glCompileShader(vertexShader);

GLuint fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);

GLuint shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);
glLinkProgram(shaderProgram);
glUseProgram(shaderProgram);

glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);

GLfloat vertices[] = {
    0.0f, 1.0f, 0.0f, 0.5f, 1.0f,

```

```

-0.3f, 0.5f, 0.0f, 0.0f, 0.5f,
 0.3f, 0.5f, 0.0f, 1.0f, 0.5f,
-1.0f, 0.0f, 0.0f, 0.0f, 0.0f,
 1.0f, 0.0f, 0.0f, 1.0f, 0.0f,
-0.3f, -0.5f, 0.0f, 0.0f, 0.5f,
 0.3f, -0.5f, 0.0f, 1.0f, 0.5f,
 0.0f, -1.0f, 0.0f, 0.5f, 0.0f,
};

GLuint indices[] = {
    0, 1, 2, 3, 4, 5, 6, 0, 6
};
GLuint VAO, VBO, EBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);
glGenBuffers(1, &EBO);
glBindVertexArray(VAO);
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, sizeof(indices), indices,
GL_STATIC_DRAW);
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 5 *
sizeof(GLfloat), (GLvoid*)0);
glEnableVertexAttribArray(0);

glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 5 *
sizeof(GLfloat), (GLvoid*)(3 * sizeof(GLfloat)));
glEnableVertexAttribArray(1);
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);
unsigned int texture = loadTexture("texture.jpg");
while (!glfwWindowShouldClose(window)) {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glm::mat4 model = glm::mat4(1.0f);
    glm::mat4 view = glm::lookAt(glm::vec3(0.0f, 0.0f, 3.0f),
glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f));
    glm::mat4 projection = glm::perspective(glm::radians(45.0f),
800.0f / 600.0f, 0.1f, 100.0f);
    GLuint modelLoc = glGetUniformLocation(shaderProgram,
"model");
    GLuint viewLoc = glGetUniformLocation(shaderProgram, "view");
    GLuint projectionLoc = glGetUniformLocation(shaderProgram,
"projection");
    GLuint lightPosLoc = glGetUniformLocation(shaderProgram,
"lightPos");

    glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
glm::value_ptr(model));
    glUniformMatrix4fv(viewLoc, 1, GL_FALSE,
glm::value_ptr(view));

```



```

        glUniformMatrix4fv(projectionLoc, 1, GL_FALSE,
glm::value_ptr(projection));
        glUniform3fv(lightPosLoc, 1, glm::value_ptr(glm::vec3(1.0f,
1.0f, 2.0f)));

        glUseProgram(shaderProgram);
        glBindVertexArray(VAO);
        glBindTexture(GL_TEXTURE_2D, texture);
        glDrawElements(GL_TRIANGLE_STRIP, 9, GL_UNSIGNED_INT, 0);

        glBindVertexArray(0);
        glfwSwapBuffers(window);
        glfwPollEvents();
    }
    glDeleteVertexArrays(1, &VAO);
    glDeleteBuffers(1, &VBO);
    glDeleteBuffers(1, &EBO);
    glfwDestroyWindow(window);
    glfwTerminate();

    return 0;
}

```

Додаток І – Прожекторне освітлення

Файл main.cpp

```

#define STB_IMAGE_IMPLEMENTATION
#include "stb_image.h"
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

// Функція завантаження текстури за допомогою stb_image
unsigned int loadTexture(const char* path) {
    unsigned int textureID;
    glGenTextures(1, &textureID);

    int width, height, nrChannels;
    unsigned char* data = stbi_load(path, &width, &height,
&nrChannels, 0);
    if (data) {
        GLenum format;
        if (nrChannels == 1)
            format = GL_RED;
        else if (nrChannels == 3)
            format = GL_RGB;
        else if (nrChannels == 4)

```

```

        format = GL_RGBA;

        glBindTexture(GL_TEXTURE_2D, textureID);
        glTexImage2D(GL_TEXTURE_2D, 0, format, width, height, 0,
format, GL_UNSIGNED_BYTE, data);
        glGenerateMipmap(GL_TEXTURE_2D);

        stbi_image_free(data);
    }
    else {
        stbi_image_free(data);
    }

    return textureID;
}

int main() {
    // Ініціалізація GLFW
    if (!glfwInit()) {
        fprintf(stderr, "GLFW initialization failed\n");
        return -1;
    }

    // Конфігурація GLFW
    glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
    glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
    glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);

    // Створення вікна GLFW
    GLFWwindow* window = glfwCreateWindow(800, 600, "SpotLight
Example", NULL, NULL);
    if (!window) {
        fprintf(stderr, "GLFW window creation failed\n");
        glfwTerminate();
        return -1;
    }

    // Налаштування контексту OpenGL
    glfwMakeContextCurrent(window);

    // Ініціалізація GLEW
    if (glewInit() != GLEW_OK) {
        fprintf(stderr, "GLEW initialization failed\n");
        glfwDestroyWindow(window);
        glfwTerminate();
        return -1;
    }

    glEnable(GL_DEPTH_TEST);
    glViewport(0, 0, 800, 600);

    // Шейдерний код вершинного шейдера

```

```

const char* vertexShaderSource = R"(
    #version 330 core
    layout (location = 0) in vec3 aPos;
    layout (location = 1) in vec2 aTexCoord;
    layout (location = 2) in vec3 aNormal;

    out vec2 TexCoord;
    out vec3 FragPos;
    out vec3 Normal;

    uniform mat4 model;
    uniform mat4 view;
    uniform mat4 projection;

    void main() {
        gl_Position = projection * view * model * vec4(aPos,
1.0);
        TexCoord = aTexCoord;
        FragPos = vec3(model * vec4(aPos, 1.0));
        Normal = mat3(transpose(inverse(model))) * aNormal;
    }
)";

// Шейдерный код фрагментного шейдера
const char* fragmentShaderSource = R"(
    #version 330 core
    out vec4 FragColor;

    in vec2 TexCoord;
    in vec3 FragPos;
    in vec3 Normal;

    struct Material {
        sampler2D diffuse;
        sampler2D specular;
        float shininess;
    };

    struct Light {
        vec3 position;
        vec3 direction;
        vec3 ambient;
        vec3 diffuse;
        vec3 specular;
        float constant;
        float linear;
        float quadratic;
        float cutOff;
        float outerCutOff;
    };

    uniform Material material;
    uniform Light light;

```

```

uniform vec3 viewPos;

void main() {
    // Ambient
    vec3 ambient = light.ambient * texture(material.diffuse,
TexCoord).rgb;

    // Diffuse
    vec3 norm = normalize(Normal);
    vec3 lightDir = normalize(light.position - FragPos);
    float diff = max(dot(norm, lightDir), 0.0);
    vec3 diffuse = light.diffuse * diff *
texture(material.diffuse, TexCoord).rgb;

    // Specular
    vec3 viewDir = normalize(viewPos - FragPos);
    vec3 reflectDir = reflect(-lightDir, norm);
    float spec = pow(max(dot(viewDir, reflectDir), 0.0),
material.shininess);
    vec3 specular = light.specular * spec *
texture(material.specular, TexCoord).rgb;

    // Attenuation
    float distance = length(light.position - FragPos);
    float attenuation = 1.0 / (light.constant + light.linear
* distance + light.quadratic * (distance * distance));

    // SpotLight
    vec3 spotDir = normalize(light.direction);
    float theta = dot(lightDir, -spotDir);
    float epsilon = light.cutOff - light.outerCutOff;
    float intensity = clamp((theta - light.outerCutOff) /
epsilon, 0.0, 1.0);

    // Final color
    vec3 result = (ambient + intensity * (diffuse +
specular)) * attenuation;
    FragColor = vec4(result, 1.0);
}

)";

// Компіляція і лінування шейдерів
GLuint vertexShader = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShader, 1, &vertexShaderSource, NULL);
glCompileShader(vertexShader);

GLuint fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShader, 1, &fragmentShaderSource, NULL);
glCompileShader(fragmentShader);

GLuint shaderProgram = glCreateProgram();
glAttachShader(shaderProgram, vertexShader);
glAttachShader(shaderProgram, fragmentShader);

```

```

glLinkProgram(shaderProgram);
glUseProgram(shaderProgram);

glDeleteShader(vertexShader);
glDeleteShader(fragmentShader);

// Вершини, текстурні координати та нормалі куба
GLfloat vertices[] = {
    // Позиції           // Текстурні координати // Нормалі
    -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,
    0.5f, -0.5f, -0.5f, 1.0f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f,
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f,
    -0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f,
    -0.5f, -0.5f, -0.5f, 0.0f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f,

    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 1.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f,
    -0.5f, 0.5f, 0.5f, 0.0f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f,
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f,

    -0.5f, 0.5f, 0.5f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, -0.5f, 1.0f, 1.0f, -1.0f, 0.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f, -1.0f, 0.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f, -1.0f, 0.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, -1.0f, 0.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, 0.5f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f, 0.0f,

    0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f,
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 1.0f, 0.0f, 0.0f, 0.0f,
    0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f, 0.0f,
    0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 1.0f, 0.0f, 0.0f, 0.0f,
    0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f,

    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f,
    0.5f, -0.5f, -0.5f, 1.0f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f,
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f, 0.0f,
    0.5f, -0.5f, 0.5f, 1.0f, 0.0f, 0.0f, -1.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, 0.5f, 0.0f, 0.0f, 0.0f, -1.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, -1.0f, 0.0f, 0.0f,

    -0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, -0.5f, 1.0f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.5f, 1.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, 0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f, 0.0f
};

// Створення буферів та об'єктів

```

```

GLuint VAO, VBO;
glGenVertexArrays(1, &VAO);
glGenBuffers(1, &VBO);

// Зв'язка об'єктів
glBindVertexArray(VAO);

glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices,
GL_STATIC_DRAW);

// Вершинні атрибути
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 9 *
sizeof(GLfloat), (GLvoid*)0);
glEnableVertexAttribArray(0);

glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 9 *
sizeof(GLfloat), (GLvoid*)(3 * sizeof(GLfloat)));
glEnableVertexAttribArray(1);

glVertexAttribPointer(2, 3, GL_FLOAT, GL_FALSE, 9 *
sizeof(GLfloat), (GLvoid*)(5 * sizeof(GLfloat)));
glEnableVertexAttribArray(2);

// Зняття зв'язку
glBindBuffer(GL_ARRAY_BUFFER, 0);
glBindVertexArray(0);

// Завантаження текстур
unsigned int diffuseMap = loadTexture("texture3.jpg");
unsigned int specularMap = loadTexture("texture4.jpg");

// Параметри світла
glm::vec3 lightPos(1.2f, 1.0f, 2.0f);
glm::vec3 lightDir(-1.0f, -1.0f, -1.0f);
glm::vec3 lightAmbient(0.8f, 0.8f, 0.8f);
glm::vec3 lightDiffuse(0.5f, 0.5f, 0.5f);
glm::vec3 lightSpecular(1.0f, 1.0f, 1.0f);
float lightConstant = 1.0f;
float lightLinear = 0.09f;
float lightQuadratic = 0.032f;
float lightCutOff = glm::cos(glm::radians(12.5f));
float lightOuterCutOff = glm::cos(glm::radians(17.5f));

// Параметри матеріалу
float materialShininess = 32.0f;

// Основний цикл рендерингу
while (!glfwWindowShouldClose(window)) {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Встановлення трансформацій та параметрів світла

```

```

//glm::mat4 model = glm::mat4(1.0f);
glm::mat4 model = glm::rotate(glm::mat4(1.0f),
glm::radians(45.0f), glm::vec3(1.0f, 1.0f, 0.0f));
glm::mat4 view = glm::lookAt(glm::vec3(0.0f, 0.0f, 2.0f),
glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 2.0f, 0.0f));
glm::mat4 projection = glm::perspective(glm::radians(45.0f),
800.0f / 600.0f, 0.1f, 100.0f);

GLuint modelLoc = glGetUniformLocation(shaderProgram,
"model");
GLuint viewLoc = glGetUniformLocation(shaderProgram, "view");
GLuint projectionLoc = glGetUniformLocation(shaderProgram,
"projection");
GLuint viewPosLoc = glGetUniformLocation(shaderProgram,
"viewPos");
GLuint lightPosLoc = glGetUniformLocation(shaderProgram,
"light.position");
GLuint lightDirLoc = glGetUniformLocation(shaderProgram,
"light.direction");
GLuint lightAmbientLoc = glGetUniformLocation(shaderProgram,
"light.ambient");
GLuint lightDiffuseLoc = glGetUniformLocation(shaderProgram,
"light.diffuse");
GLuint lightSpecularLoc = glGetUniformLocation(shaderProgram,
"light.specular");
GLuint lightConstantLoc = glGetUniformLocation(shaderProgram,
"light.constant");
GLuint lightLinearLoc = glGetUniformLocation(shaderProgram,
"light.linear");
GLuint lightQuadraticLoc =
glGetUniformLocation(shaderProgram, "light.quadratic");
GLuint lightCutoffLoc = glGetUniformLocation(shaderProgram,
"light.cutoff");
GLuint lightOuterCutoffLoc =
glGetUniformLocation(shaderProgram, "light.outerCutoff");
GLuint materialShininessLoc =
glGetUniformLocation(shaderProgram, "material.shininess");

glUniformMatrix4fv(modelLoc, 1, GL_FALSE,
glm::value_ptr(model));
glUniformMatrix4fv(viewLoc, 1, GL_FALSE,
glm::value_ptr(view));
glUniformMatrix4fv(projectionLoc, 1, GL_FALSE,
glm::value_ptr(projection));
glUniform3fv(viewPosLoc, 1, glm::value_ptr(glm::vec3(0.0f,
0.0f, 3.0f)));
glUniform3fv(lightPosLoc, 1, glm::value_ptr(lightPos));
glUniform3fv(lightDirLoc, 1, glm::value_ptr(lightDir));
glUniform3fv(lightAmbientLoc, 1,
glm::value_ptr(lightAmbient));
glUniform3fv(lightDiffuseLoc, 1,
glm::value_ptr(lightDiffuse));

```

```

        glUniform3fv(lightSpecularLoc, 1,
glm::value_ptr(lightSpecular));
        glUniform1f(lightConstantLoc, lightConstant);
        glUniform1f(lightLinearLoc, lightLinear);
        glUniform1f(lightQuadraticLoc, lightQuadratic);
        glUniform1f(lightCutOffLoc, lightCutOff);
        glUniform1f(lightOuterCutOffLoc, lightOuterCutOff);
        glUniform1f(materialShininessLoc, materialShininess);

        // Використання текстур
        glUseProgram(shaderProgram);
        glBindVertexArray(VAO);
        glBindTexture(GL_TEXTURE_2D, diffuseMap); // Зв'язка
текстур

        // Малювання куба
        glDrawArrays(GL_TRIANGLES, 0, 36);

        glBindVertexArray(0);

        // Перевірка та обробка подій GLFW
        glfwSwapBuffers(window);
        glfwPollEvents();
    }

    // Вивільнення ресурсів
    glDeleteVertexArrays(1, &VAO);
    glDeleteBuffers(1, &VBO);

    // Завершення роботи GLFW
    glfwDestroyWindow(window);
    glfwTerminate();

    return 0;
}

```


РЕКОМЕНДОВАНІ ДЖЕРЕЛА ІНФОРМАЦІЇ

1. Gordon, V. S. Computer Graphics Programming in OpenGL with C++/ V. S. Gordon, J. Clevenger. - 2nd ed. Publisher: David Pallai Mercury Learning and Information, 2021. - 535 p.
2. OpenGL, <https://uk.wikipedia.org/wiki/OpenGL>
3. OpenGL 3.3. Новий стандарт, <https://www.youtube.com/playlist?list=PLlaa7-TIzBOhnXPiRT4DuV2XSoPTB5BAx>
4. OpenGL. Вступ, <https://ppt-online.org/594034>
5. Візуальне програмування, <http://surl.li/msfsg>
6. Графічна бібліотека, <http://surl.li/msfsl>